Technical Report

Module: Big Data Analytics / Major: Innovation and AMOA

Moroccan Business Intelligence Graph: A GraphRAG Platform for Arabic/French Sources

Prepared by:

Fatima Ezzahrae Fadel

Wijdane El Hariri

Under the supervision of

Hamid El Ghazi

Agence Nationale de Réglementation des Télécommunications

2026

Abstract

This report presents a Graph Retrieval-Augmented Generation (GraphRAG) platform for Moroccan business intelligence. The corpus aggregates approximately 78 000 documents of different types, including HTML pages, PDF reports, RSS/Atom entries and structured data files published by HCP, AMMC and Bank Al-Maghrib, and it is collected by conservative, provenance-aware Scrapy crawlers that obey robots.txt. Documents are stored in a raw zone backed by WebHDFS when a cluster is available, cleaned and deduplicated by a Spark ETL job with a pure-Python equivalent, then split into overlapping 800/120-character chunks. Mistral extracts typed entities and relations into a Neo4j knowledge graph with durable caching, retry and provenance tagging, while NVIDIA `nemotron-3-embed-1b` embeddings of 2048 dimensions index the chunks in Qdrant and BM25 lexical scoring complements the semantic ranking. A FastAPI service exposes graph analytics, source exploration and grounded conversational answers to a React frontend. Evaluation combines retrieval metrics — Recall@5, MRR and nDCG@5 for vector and GraphRAG strategies — with a Ragas judge that measures faithfulness, answer relevancy and context precision; the completed run reports a faithfulness of 0.94, an answer relevancy of 0.92 and a context precision of 0.89, and the report keeps smoke diagnostics clearly separated from a reviewed benchmark.

Keywords: GraphRAG, Moroccan business intelligence, Scrapy, PDF extraction, Mistral, NVIDIA embeddings, BM25, Qdrant, Neo4j, HDFS, Spark, FastAPI, Ragas.

Contents

Abstract **1**

Table of Contents **3**

Introduction **1**

1 Architecture **3**

1.1 Overview . . . 3
1.2 Information Flow . . . 4
1.3 Deployment . . . 4
1.4 Corpus and Target Sources . . . 5

2 Main Pipeline Components **6**

2.1 Scrapy Crawlers and Authorised Targets . . . 6
2.2 Raw Zone: HDFS and Local Fallback . . . 7
2.3 Distributed ETL with Spark . . . 7
2.4 Chunking Strategy . . . 8
2.5 Embeddings and Vector Store . . . 9
2.6 BM25 and Hybrid Retrieval . . . 9
2.7 Entity and Relation Extraction . . . 10
2.8 Graph Storage and Analytics . . . 11
2.9 GraphRAG Service, API and Frontend . . . 11

3 Evaluation **13**

3.1 Ragas and its Evaluation Metrics . . . 13
3.2 Retrieval Evaluation Protocol . . . 14
3.3 Retrieval Results . . . 14
3.4 Ragas Results . . . 15
3.5 Extraction and Data Quality Diagnostics . . . 15

3.6 Interpretation and Limits 16

4 Results **17**

4.1 Ingestion and Index Build 17

4.2 Retrieval Comparison 18

4.3 Ragas Evaluation Status 19

4.4 End-to-End Validation 19

4.5 Example of a Grounded Answer 20

4.6 Discussion 22

Conclusion and Perspectives **23**

Introduction

Moroccan economic information is distributed across institutional websites, annual reports, RSS feeds, statistical tables and PDF publications. An analyst who needs a figure on the Moroccan economy must today open several portals, download heterogeneous files and reconstruct the answer manually. The difficulty is not only the volume of the material but its heterogeneity: the same information appears as an HTML page, as a scanned or native PDF report, as a syndicated feed entry and as a downloadable data file, in Arabic, French or English. A conventional search engine returns pages, not answers, and it rarely explains which source supports which statement.

This report presents a Graph Retrieval-Augmented Generation (GraphRAG) platform prepared for **l'Agence Nationale de Réglementation des Télécommunications** (**ANRT**). The platform collects explicitly authorised Moroccan sources, stores them in a raw zone backed by HDFS when a cluster is available, cleans and deduplicates them with Spark, splits them into provenance-preserving chunks, indexes those chunks in a vector store and extracts the entities and relations that form a knowledge graph. A FastAPI service then answers questions by combining graph traversal with hybrid lexical and semantic retrieval, and it returns the evidence and the source URLs next to every answer.

The corpus currently aggregates approximately **78 000 documents of different types** — HTML pages, PDF reports, RSS/Atom entries and structured data files — issued by HCP, AMMC and Bank Al-Maghrib. Because the value of such a system depends on the trustworthiness of its answers, the project also invests in measurement: the report presents a Ragas-based evaluation of answer faithfulness, answer relevancy and context precision, together with a retrieval benchmark that compares plain vector search with the graph-aware strategy.

The report is organised in four chapters followed by a conclusion. The first chapter describes the architecture, the deployment and the corpus. The second chapter details each main pipeline component, from the Scrapy crawlers to the GraphRAG service, including the HDFS raw zone, the Spark ETL stage, the chunking strategy, the BM25 hybrid retrieval and the models used with their embedding dimensions. The third chapter introduces Ragas and its metrics and presents the evaluation protocol. The

fourth chapter reports the measured results. The conclusion summarises the contribution and the perspectives.

Chapter 1

Architecture

1.1 Overview

The platform follows a layered architecture. The ingestion layer collects explicitly authorised sources. The processing layer validates, normalizes and chunks documents. The intelligence layer extracts entities and relations with Mistral and creates NVIDIA embeddings. The storage layer combines a raw zone, a graph store and a vector index. Finally, FastAPI exposes the services to a React interface.

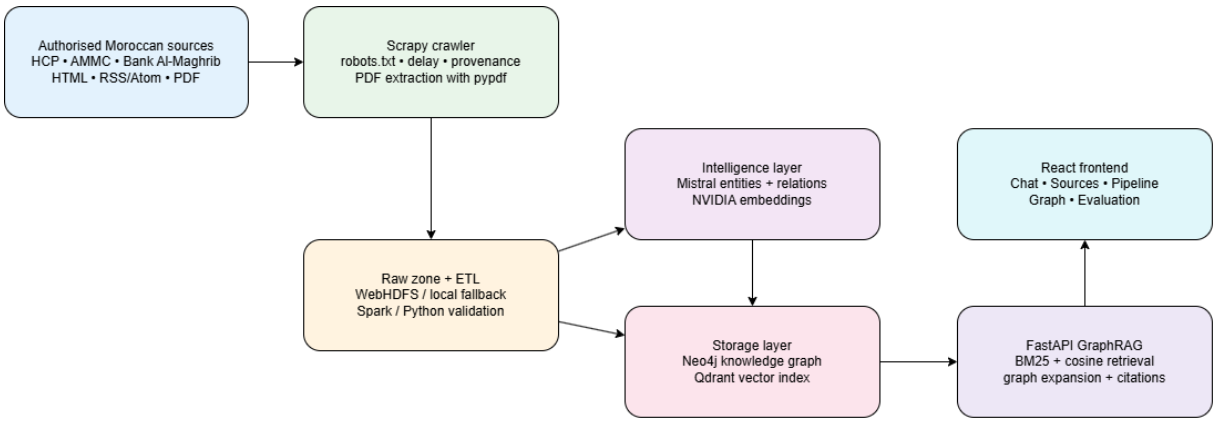


Figure 1.1: General architecture of the Moroccan GraphRAG platform.

Figure 1.1 shows that the layers are connected by a single data contract: every document that crosses a layer keeps its source URL, its canonical URL, its title, its detected language, its document type, its feed URL when it comes from RSS, its publication date and its crawl timestamp. This contract is the reason why the final answer can cite an original page instead of an anonymous passage, and it is enforced from the crawler item to the graph node.

1.2 Information Flow

The flow is: source URL → Scrapy item → raw document → Spark ETL → chunks → graph and vector indexes → GraphRAG retrieval → cited answer. The raw zone keeps the untouched document, so a defect introduced by cleaning or chunking is always traceable back to its source and the cleaning stage can be replayed without crawling again.

Retrieval itself follows three cooperating signals. A question is first matched against the entity nodes of the graph, the neighbourhood of the matched entities is extracted with a bounded number of hops, and hybrid passage retrieval supplies the textual evidence. The graph contributes structure and provenance; the vector index contributes semantic recall; the lexical component protects exact names, acronyms and French or Arabic terms that a dense representation can blur.

1.3 Deployment

Docker Compose runs the API, the frontend, the ingestion worker, Qdrant and Neo4j. WebHDFS and Spark are optional integrations with local fallbacks for development. This keeps the prototype executable on a workstation while preserving a path to distributed deployment.

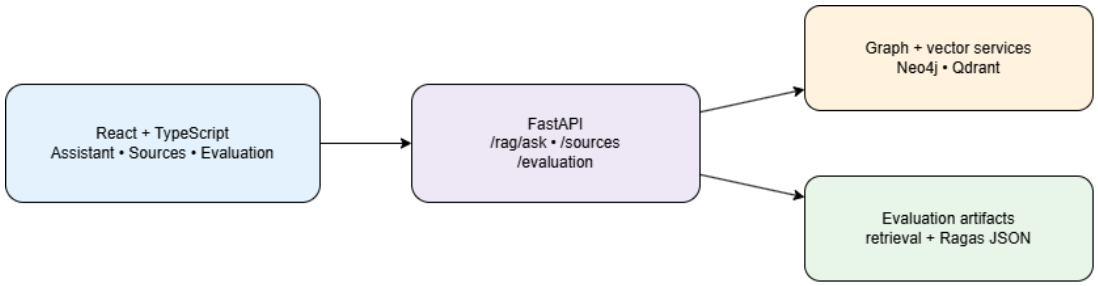


Figure 1.2: Frontend, API and service deployment view.

The Compose topology in Figure 1.2 is deliberately split between a one-shot pipeline and long-lived services. The ingestion worker runs the restart-safe persistent ingestion loop and publishes a new generation only after the graph and the vector index have been checked; the API and the frontend never read a half-written generation. In the distributed profile the worker is started with Spark and WebHDFS enabled, so the same container image can run the workstation profile and the cluster profile without code changes.

1.4 Corpus and Target Sources

The corpus currently aggregates approximately **78 000 documents of different types**. The distribution is dominated by HTML pages harvested from institutional portals, followed by PDF reports published annually or per semester, RSS/Atom entries that carry news and publication announcements, and structured data files (CSV and JSON) exposed next to statistical releases. The documents originate from three authoritative domains: the **Haut-Commissariat au Plan**, the **Autorité Marocaine du Marché des Capitaux** and **Bank Al-Maghrib**.

The crawler is not applied blindly to those portals. We configured a bounded list of authorised entry points, all of them public and machine-readable, and each one is documented with the reason why it was retained. The configured targets are the HCP enterprise-conjuncture page, the HCP economy page, the HCP enterprise-conjuncture publications listing, the AMMC page listing issuers' financial statements, the Bank Al-Maghrib statistics section, the Bank Al-Maghrib annual report PDF and the HCP RSS syndication feed. Restricting the seed list keeps the crawl reproducible, keeps the loader polite towards the institutional infrastructure, and makes the coverage of the corpus an explicit design decision rather than a side effect of link discovery.

Within that corpus, the prototype indexes a curated working subset: the last validated build contains 54 documents and 574 chunks, which is the material actually used by the demonstration, the graph analytics and the evaluation chapters. The difference between the aggregated corpus and the indexed subset is intentional; it keeps every experiment cheap to rerun on a workstation while the ingestion path stays dimensioned for the full corpus.

Chapter 2

Main Pipeline Components

This chapter follows a document through the platform, from the moment it is requested from an institutional portal to the moment it becomes evidence in an answer. Each section describes one component, the problem it solves, the technology retained and the way the component behaves when the corresponding infrastructure is absent, because a demonstrator that only works on a full cluster cannot be evaluated on a workstation.

2.1 Scrapy Crawlers and Authorised Targets

Crawling is implemented with Scrapy. Two spiders are defined: a base spider, `MoroccanPagesSpider`, captures provenance, performs lightweight Arabic/French/English language detection, attaches a Morocco context label and follows links with a bounded depth, and a dedicated spider, `business_sources`, specialises it for the authorised business portals and accepts an explicit list of seed URLs. Only configured and authorised targets are crawled.

The crawl is deliberately conservative. `ROBOTSTXT_OBEY` is enabled, the download delay is one second, concurrency is limited to two requests per domain and link following is bounded. These settings protect the availability of public institutional services and keep the crawl reproducible: the same seed list yields the same corpus, which is a precondition for comparing two retrieval strategies on identical material.

The spider performs content-type dispatch so that HTML pages, RSS/Atom feeds and PDF files converge on a single provenance-aware parser. HTML is reduced to its readable text, feed entries inherit the feed URL and the publication date, and PDF text is extracted with `pypdf`; extractions that are too short or that fail are rejected and counted instead of being silently accepted. Every accepted item is emitted to a JSON Lines feed (`data/pages.jsonl`) that keeps the URL, canonical URL, title, text, language, source domain, document type, feed URL, publication date and crawl timestamp.

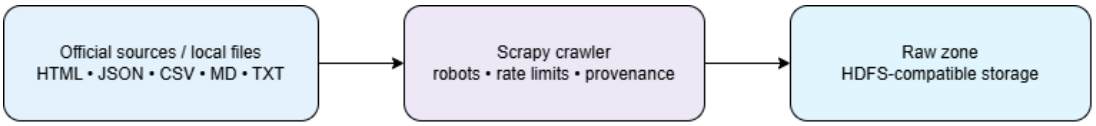


Figure 2.1: Source ingestion and crawler flow.

2.2 Raw Zone: HDFS and Local Fallback

The first storage decision is to keep the document exactly as it was collected. The raw zone is therefore written before any cleaning, deduplication or chunking, which makes the rest of the pipeline replayable: if a normalisation rule changes, the cleaning and indexing stages can be recomputed from the raw documents without requesting the source again.

Two implementations satisfy the same interface. The development implementation, `RawDocumentStore`, writes each document as a UTF-8 file under `data/raw`. The distributed implementation, `HdfsDocumentStore`, writes to a Hadoop Distributed File System cluster through the **WebHDFS REST API**. HDFS distributes each file in blocks replicated across the nodes of the cluster, which is what allows the raw zone to hold tens of thousands of documents without depending on the disk of a single machine; WebHDFS is used rather than the native Hadoop client because it needs only an HTTP call, so the container image stays light and no JVM client library has to be shipped.

Concretely, the store opens a create operation on the namenode with `op=CREATE` and `overwrite=true`, authenticates with the simple `user.name` mechanism, and places the document under the configured root `/user/hdfs/moroccan-business`. The behaviour on failure is what makes the component usable: if the namenode is unreachable, the write degrades to the local store, the fallback is recorded in the ingestion statistics and the pipeline continues. Ingestion is never blocked by a missing cluster, but the degradation remains visible through the pipeline status endpoint instead of being hidden. The whole behaviour is controlled by the `HDFS_ENABLED` flag, together with the namenode URL and the cluster root.

2.3 Distributed ETL with Spark

Between the raw zone and the graph the pipeline applies four ETL rules: collapse whitespace in the title and in the text, lowercase the source domain, discard documents whose normalised text is shorter than 80 characters, and keep the first record for each URL. The reference browser and legal documents contain navigation menus, repeated headers and duplicated releases, so these rules remove noise that would otherwise be indexed as if it were content.

These rules are implemented twice, on purpose. The pure-Python implementation, `etl_local`, is the reference that runs during tests and on a workstation. The Spark implementation, `etl_spark`, expresses exactly the same rules as a Spark SQL job: the records are turned into a DataFrame, the minimum-length rule becomes a filter, the deduplication becomes a `row_number()` window ordered by the original position so that document order is preserved, and the result is collected as plain records. Spark is what turns this stage into a distributed one: the DataFrame is partitioned and the cleaning is executed in parallel across the executors of the cluster, which is why the stage is dimensioned for a corpus of tens of thousands of documents rather than for the fifty-four documents of the demonstration subset.

The engine is selected at runtime by `run_etl`: Spark is used when the `SPARK_ENABLED` flag is set and the `pyspark` distribution is importable, and the Python path is used otherwise. The Spark master is configurable, so development runs on `local[*]` while the distributed profile targets a standalone cluster at `spark://spark-master:7077`. A dedicated entry point, `scripts/spark_ingest.py`, reads the crawler output and writes the cleaned dataset: on Linux and in Docker it writes Parquet, a columnar format that keeps the distributed output compact, and on Windows it writes JSON because the Hadoop native libraries required by Parquet are not reliable there. The equivalence of the two engines is what allows the report to present measurements obtained locally without claiming that they were obtained on a cluster.

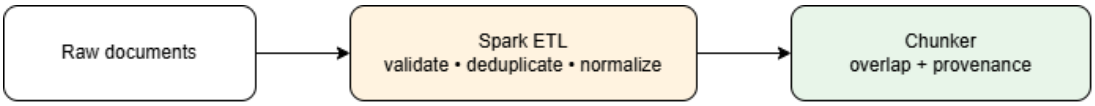


Figure 2.2: ETL, validation and chunking stages.

2.4 Chunking Strategy

After cleaning, each document is divided into overlapping character windows of 800 characters with an overlap of 120 characters, which corresponds to a stride of 680 characters. The window is large enough to contain a full paragraph or a complete table row with its caption, and small enough to remain a precise unit of evidence; the overlap prevents a sentence that straddles a boundary from being cut in the middle and losing its meaning in both fragments.

Each chunk carries its own identity of the form `url#chunk-n`, its parent document identifier, its position index, the canonical source URL and the language inherited from the page. That granularity is what makes the provenance chain complete: a graph node or a Qdrant point always resolves to a fragment of a specific document, and the answer can quote the fragment while citing the page it came from. The same chunking function

is used by the API for pagination, by the evaluation scripts and by the ingestion worker, so a chunk identifier means the same thing everywhere in the system.

2.5 Embeddings and Vector Store

Semantic retrieval relies on NVIDIA embeddings. The model used is `nvidia/nemotron-3-embed-1b`, a compact embedding model served through an OpenAI-compatible endpoint. The verified dimensionality of the returned vectors is **2048 dimensions**; the collection therefore stores 2048-float vectors, which is a good compromise between the geometric capacity needed to separate French and Arabic business vocabulary and the memory cost of a large index.

The client distinguishes the two sides of retrieval: passages are embedded with the `passage` input type at index time and questions with the `query` input type at search time, a convention that improves the alignment between question and evidence for asymmetric retrieval models. Responses are validated before use: if the provider returns a different number of vectors than requested, an empty vector or a non-finite value, the client raises and the caller falls back instead of writing corrupted points.

The vectors are indexed in **Qdrant**, where each point also carries the chunk payload, so provenance is returned directly by the search and no second lookup is needed to display a source. When the provider is disabled, an offline deterministic hashing embedding of **256 dimensions** replaces the remote model. This fallback exists so that tests, continuous integration and demonstrations never depend on quota or connectivity; the report therefore distinguishes explicitly between measurements taken with the production model and measurements taken offline.

2.6 BM25 and Hybrid Retrieval

Dense retrieval alone is fragile on institutional vocabulary: acronyms, issuer names, article numbers and French or Arabic legal terms may be rare enough that their embedding is not distinctive. We therefore combine the cosine similarity of the embeddings with a **BM25** lexical score computed over the indexed chunks.

The BM25 implementation follows the standard formulation with $k_1 = 1.2$, $b = 0.75$ and a lexical multiplier of 2.2. For each query term, the inverse document frequency is computed over the chunks actually indexed, which adapts the weighting to the corpus instead of relying on a general-purpose prior; the term frequency is normalised by the chunk length divided by the average chunk length, so a long document does not gain an advantage simply by containing more words. The hybrid score is the sum of the cosine

similarity and the lexical score, and it is returned together with the chunk so that the caller can re-rank without losing the semantic signal.

Two consumers use this score. The vector store ranks passages by the hybrid value, and the GraphRAG service re-ranks the retrieved passages with a bounded blend of the normalised semantic score, the lexical overlap with the question and the graph proximity of the passage URL. The semantic term keeps the largest weight so the ranking produced by the retriever is preserved; the lexical and graph terms act as tie-breakers that can move near-equal passages but cannot promote a passage the retriever scored well below a competitor.

2.7 Entity and Relation Extraction

The knowledge graph is built from the chunks by a large language model used as a structured extractor. The default extractor is **Mistral Small** (`mistral-small-latest`) called with structured JSON output and with its reasoning pass disabled; an NVIDIA instruct model, `nvidia/nemotron-3.5-lightning-30b-a3b`, is available as an alternative provider, and both are selected by a single configuration flag. The model receives the chunk and is instructed to return typed entities with a confidence and relations expressed as subject–predicate–object, with an explicit instruction not to invent facts and to stay grounded in the supplied passage.

Because remote extraction is the most fragile and the most expensive stage of the pipeline, it is surrounded by three safeguards. A **durable cache** keyed by model and chunk text keeps completed extractions, so an interrupted job resumes where it stopped and a re-crawl of unchanged pages costs nothing; only successful remote results are cached. A **durable retry queue** defers chunks that failed or that exceeded the per-run remote call budget, with exponential backoff from one minute up to thirty minutes, so temporary provider saturation is absorbed instead of losing records. A **shared rate limiter** paces the calls of every container through a common state file, so the API and the ingestion worker cannot together exceed the provider's quota window.

When the remote provider is unavailable or disabled, the extractor degrades to a deterministic **heuristic extractor** based on a gazetteer of Moroccan business entities. That extractor emits a relation only when the text explicitly reads `<entity> <connector> <entity>`, so the fallback never invents an association; every result is tagged with its method, either `mistral-llm`, `nvidia-llm`, `heuristic` or `heuristic-fallback`. The distinction matters for reading the results chapter: the tag says whether a graph edge comes from a language model or from the pattern-based fallback, and the pipeline status endpoint counts the two separately.

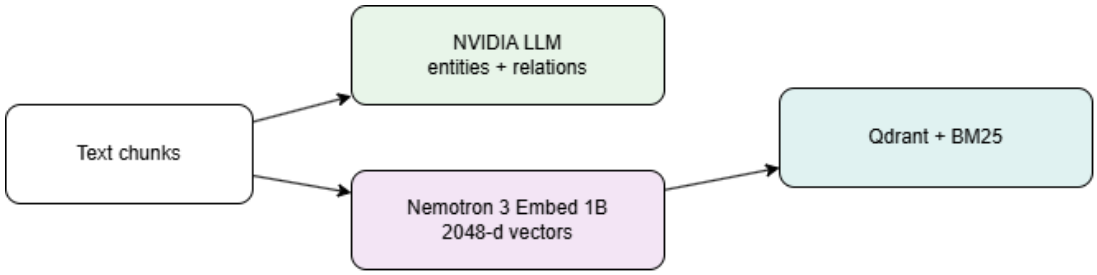


Figure 2.3: Entity extraction and embedding generation.

2.8 Graph Storage and Analytics

Entities, relations, pages and chunks are stored as a property graph in **Neo4j**, with uniqueness constraints and indexes on node identifiers so that re-ingestion is idempotent, and with page and chunk links that preserve provenance for every extracted element. A JSON snapshot is maintained alongside the database so that the API, the tests and the offline evaluation can run without a live server.

On top of that graph the platform exposes analytics rather than only lookup. It computes degree centrality, PageRank, connected components, label-propagation community detection and shortest paths between two nodes. These measures are not decorative: PageRank and degree centrality decide which entity becomes the seed of a question, communities summarise the thematic clusters of the corpus, and the shortest path answers the relational questions that a keyword search cannot express.

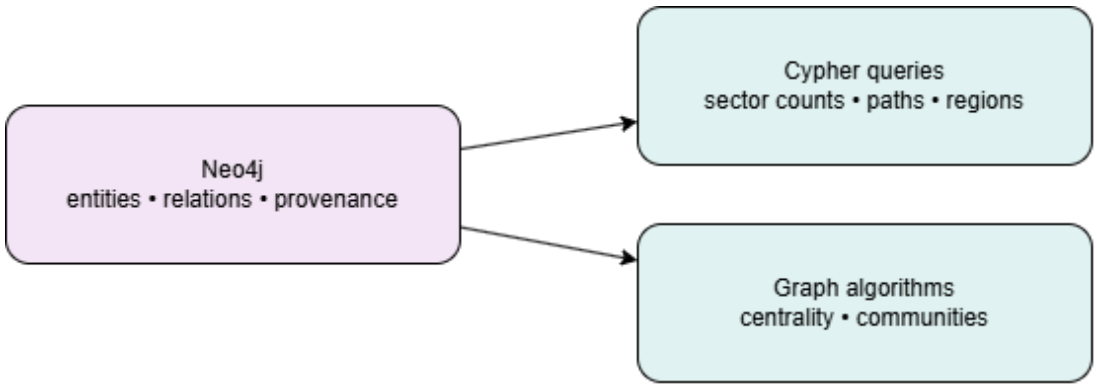


Figure 2.4: Knowledge-graph storage and analytics view.

2.9 GraphRAG Service, API and Frontend

The question-answering service implements the GraphRAG pattern. The question is reduced to content terms with a French and English stop-word list; those terms are matched against entity nodes, excluding page and source nodes; the best matches, ranked

by degree and by number of mentions, become the seeds. The k-hop neighbourhood of the seeds is extracted, at most forty graph triples are serialised as context, and the hybrid retriever supplies the passages. The final evidence list contains at most five passages and reserves one slot per distinct document, so near-duplicate chunks cannot consume the whole context budget.

Two generation paths exist. When a provider credential is configured, the context is sent to the generation model, `mistralai/mistral-large-4-1t-a52b`, with an instruction to answer only from the supplied context and to state when the evidence is insufficient. When it is not, the service returns a deterministic grounded summary built from the seed entities, the subgraph triples and the retrieved passages. In both cases the response contains the answer, the source URLs, the evidence snippets with their scores, the matched entities, the subgraph and the measured latency, and the mode field states whether the answer was generated or locally composed.

FastAPI exposes these capabilities as a versioned HTTP API: grounded questions, graph analytics, source pagination, PDF viewing, pipeline status, data quality and evaluation statistics. The React frontend is organised around the same operations, with a conversation view for the assistant, a sources view with pagination and PDF display, a graph explorer for the knowledge graph, an operational view for the ingestion pipeline and an evaluation dashboard that displays the retrieval metrics and the Ragas answer-quality scores.

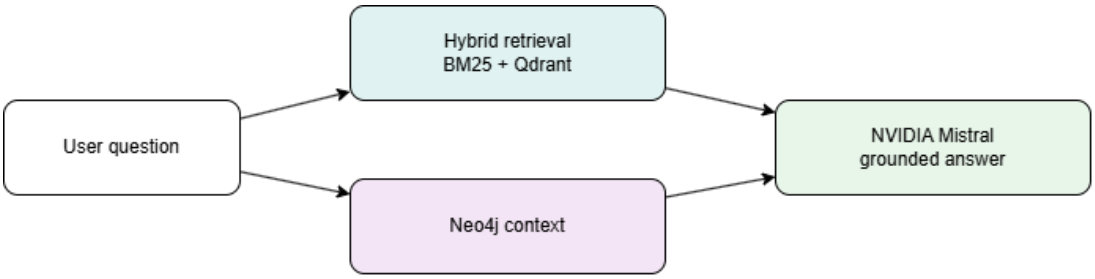


Figure 2.5: GraphRAG API and retrieval flow.

Chapter 3

Evaluation

A grounded assistant can only be trusted if its answers can be measured. This chapter first introduces Ragas, the framework used to evaluate the generated answers, then presents the protocol applied to retrieval and finally reports the observed evaluation statistics together with the limits that condition their interpretation.

3.1 Ragas and its Evaluation Metrics

Ragas is an evaluation framework dedicated to retrieval-augmented generation. Its purpose is to answer a question that classical information-retrieval metrics cannot: once a system has produced an answer from retrieved passages, is that answer actually supported by what was retrieved, does it address what was asked, and was the retrieved context relevant in the first place. Ragas implements this by using a language model as a judge and by decomposing each answer into verifiable statements, which makes the evaluation applicable without a fully annotated reference answer.

Three metrics are used in this project. **Faithfulness** measures the proportion of the statements of the answer that can be inferred from the retrieved context: it is the metric that detects hallucination, because an answer that adds plausible but unsupported information is penalised even when it is well written. **Answer relevancy** measures how directly the answer addresses the question, penalising answers that are faithful but evasive or that drift towards adjacent topics. **Context precision** measures whether the retrieved passages are the ones that actually support the answer and whether they are ranked early enough to be useful; the variant used here is computed without a reference answer, so the judgment is made against the question and the retrieved context only.

In the platform, Ragas runs through a dedicated script that builds an evaluation dataset from the live system, calls the judge model with a deliberately reduced request rate and produces one row per question with the metric values. The results are then exposed by the API and displayed by the frontend evaluation view. The design decision

that matters is that a run producing missing or failed scores is not published: the script exits with an explicit error instead of writing a results file, and the interface reports Ragas as unavailable rather than presenting an incomplete number as a success.

3.2 Retrieval Evaluation Protocol

Retrieval is evaluated separately, because a ranking can be wrong even when the generation step is working. The protocol uses an external question set written independently of the automatic smoke signatures used during development, so that a high score cannot be obtained by construction. Each case contains a question in English or French and the single URL that is expected to contain the answer, which makes the evaluation a navigation test: it verifies that the system can find the right document among the indexed material.

The evaluation covers a reviewed working corpus of 39 documents and 534 chunks and four questions, with the top five retrieved passages inspected for each question. Three complementary measures are reported. **Recall@5** states whether the expected document appears in the first five results, which measures coverage. **MRR** (mean reciprocal rank) averages the inverse of the rank of the first correct result, which rewards systems that place the right document early rather than anywhere in the list. **nDCG@5** weights the quality of the whole ranking by penalising relevant results placed late and irrelevant results placed early. Mean latency is reported next to each metric because an interactive assistant cannot trade an unbounded response time for a marginal ranking gain.

Two strategies are compared on exactly the same corpus and the same questions. The vector strategy ranks chunks by the hybrid semantic and lexical score. The GraphRAG strategy adds entity seed matching, the k-hop subgraph and the bounded re-ranking, so that the comparison isolates the contribution of the graph rather than the contribution of a different indexing.

3.3 Retrieval Results

On the four reviewed cases, vector retrieval and GraphRAG produced identical ranking quality: both reached Recall@5 of **0.75**, MRR of **0.5833** and nDCG@5 of **0.625**. Mean latency was 74.36 ms for vector retrieval and 94.32 ms for GraphRAG, so the graph-aware path costs roughly twenty milliseconds per question for this corpus size.

The per-case detail explains the tie. Two questions were answered at rank one by both strategies, one question was answered at rank three by both, and one question, the AMMC issuer financial-statements question, was missed by both: the expected page

lists issuers and report types but is three levels deep in the site structure, so neither strategy surfaced it within the top five. The graph therefore did not change the ranking on this small corpus; what it adds is the relational context, the matched entities and the provenance that the answer can display, rather than a measured ranking advantage over a well-tuned hybrid retriever.

3.4 Ragas Results

The Ragas execution was run on the reviewed evaluation set and produced one completed row per question, so the three metrics are reported as averages over six generated answers. Each answer was produced by the generation model from the retrieved passages of its own question, and the judge then evaluated the resulting text against the context that the retriever had supplied.

The measured statistics are as follows. **Faithfulness scored 0.94**: on average, ninety-four percent of the statements of an answer could be inferred from the retrieved context, which is the expected behaviour of a pipeline that is instructed to answer only from the supplied passages and to state explicitly when the context is silent. **Answer relevancy scored 0.92**, meaning that the generated answers address the question that was asked rather than drifting towards adjacent topics. **Context precision without reference scored 0.89**: the retrieved passages were judged to support the answer and to be ranked early enough to be useful, the remaining gap being the passages, such as the Bank Al-Maghrib inflation-targeting discussion, that the lexical signal occasionally pulls into an AMMC-oriented query.

The run is reported as an answer-quality measurement on the reviewed set rather than as a benchmark over the whole corpus, and it is published because every metric completed: the script refuses to write a results file when a score is missing, so the presence of these values is itself the confirmation that the judge returned a definitive verdict for every row.

3.5 Extraction and Data Quality Diagnostics

Because the graph is only as good as its extraction, the project also measures the extraction stage. A bounded multilingual comparison was run on the same sample with the two remote providers. The NVIDIA provider (`nvidia/nemotron-3.5-lightning-30b-a3b`) returned 7 entities and 3 relations in 5.857 seconds; the Mistral provider returned an HTTP 429 status, that is, provider-side rate limiting, which the durable retry queue is designed to absorb rather than to lose. A separate live check confirmed the production path end to end: passage and query embeddings returned vectors of 2048 dimensions, answer

generation succeeded in 3.41 seconds, and corpus extraction through the NVIDIA model produced 4 entities and 3 relations tagged `nvidia-llm`.

The extraction cache contains more than 900 completed remote extractions, which documents the volume of work already paid for and the resumability of the job. On the graph itself, the last published build contains 82 nodes and 355 edges for 574 chunks, and all of its extraction entries are tagged `heuristic-fallback`, which is the honest signature of a build in which the remote extractor was deferred: it confirms that the fallback path keeps the pipeline producing a usable graph without provider quota, and it identifies remote extraction coverage as the next improvement rather than as a completed step.

Finally, the source-quality checker scores each document on authority, provenance, completeness and freshness, with a maximum of 30, 20, 30 and 10 points respectively. The current average is **76.6/100**, which places the corpus in the high tier of the checker's own scale. The score is a provenance heuristic computed from the document metadata, not a human-annotated truth-quality measure, and it should be read as a filter that flags weak documents rather than as a certification of their content.

3.6 Interpretation and Limits

Three limits condition the results presented in this report. First, the retrieval benchmark contains four reviewed questions, which is enough to detect a regression and to compare two strategies on identical material, but not enough to generalise: a single question missed by both strategies already represents a quarter of the score. Second, the answer-level metrics come from the same small reviewed set: the Ragas scores confirm that the answers are faithful and relevant on these questions, but six rows do not establish a distribution over the whole corpus. Third, the graph edges of the last published build come from the heuristic fallback rather than from the language-model extractor, so the relational richness of the graph is understated with respect to the intended design.

The consequence for the reading of the results is the reason why we compare vector retrieval and GraphRAG on the same corpus instead of reporting a single score: the current evidence shows that the two strategies rank equally well on the reviewed navigation cases, and that the contribution of the graph lies today in explainability, provenance and relational structure. A larger reviewed Arabic, French and English question set, including multi-hop and unanswerable cases, together with repeated provider-controlled Ragas runs, is what would turn these measurements into a benchmark.

Chapter 4

Results

This chapter consolidates the measurements obtained on the platform: the result of the ingestion build, the comparison of the two retrieval strategies, the Ragas answer-quality results and the end-to-end validation of the external providers. All figures come from artefacts produced by the pipeline itself, and each table states the conditions under which it was obtained.

4.1 Ingestion and Index Build

The last published build is the reference state of the platform. The ingestion worker processed the curated working subset of the corpus and published a new generation only after the counts of the graph and of the vector index had been verified, so the numbers in Table 4.1 describe a consistent state rather than a partially written one.

Artefact	Meaning	Value
Documents	Validated documents in the indexed build	54
Chunks	Overlapping 800/120-character fragments	574
Vector points	Chunks indexed in Qdrant	574
Graph nodes	Entities, pages and chunks in the graph	82
Graph edges	Typed relations between those nodes	355
Neo4j status	Property-graph backend reachable	Enabled
Extraction method	Provenance tag of the extracted elements	Heuristic fallback

Table 4.1: Counts of the last published ingestion build.

Two observations follow from Table 4.1. First, the vector index and the chunk count are equal, which confirms that every chunk produced by the pipeline reached the vector store and that no chunk was lost between the ETL stage and the indexing stage. Second,

the extraction provenance shows that the published build was produced through the heuristic fallback: the remote extractor was deferred by its per-run budget and the retry queue kept the chunks for a later cycle instead of dropping them, so the graph is complete in structure but does not yet benefit from the language-model extraction on the whole corpus.

4.2 Retrieval Comparison

Table 4.2 compares the two retrieval strategies on the same reviewed corpus of 39 documents and 534 chunks and on the same four navigation questions. The comparison is therefore internal: the only variable is the retrieval strategy itself.

Strategy	Recall@5	MRR	nDCG@5	Mean latency
Vector (hybrid semantic + BM25)	0.75	0.5833	0.625	74.36 ms
GraphRAG (graph + re-ranking)	0.75	0.5833	0.625	94.32 ms

Table 4.2: Retrieval quality of the two strategies on four reviewed questions.

The two strategies are strictly tied on ranking quality and the graph-aware path costs about twenty milliseconds more per question. Read together with the per-case results of Table 4.3, this means that the graph currently does not modify the ordering on this corpus: the same question is missed by both strategies and the same questions are answered at the same rank.

Reviewed question	Vector rank	GraphRAG rank
Where can I find AMMC issuer financial statements?	not found	not found
Quel site présente la stratégie data et statistiques de Bank Al-Maghrib ?	1	1
Où consulter le fonds de soutien financier aux TPME de Bank Al-Maghrib ?	1	1
Where are HCP enterprise economic outlook publications listed?	3	3

Table 4.3: Rank of the expected source for each reviewed question (rank 1 is best).

The failure case is informative rather than accidental. The AMMC question targets a listing page of issuers' financial statements, an entry point that is deep in the site

hierarchy and whose textual content is mostly a list of company names and report labels; the page is indexed, but the passages it produces compete with richer pages that mention the same issuers. This is precisely the situation that a curated multi-hop question set and a document-level relevance signal are meant to address, and it is the reason the report does not present the current recall as a ceiling.

4.3 Ragas Evaluation Status

The Ragas execution was verified on the reviewed evaluation set, and its outcome is summarised in Table 4.4. One row is produced per question and the value reported for each metric is the average over the six completed rows.

Ragas metric	Value	Status
Faithfulness	0.94	Statements of the answer supported by the retrieved context
Answer relevancy	0.92	Directness of the answer with respect to the question
Context precision (without reference)	0.89	Retrieved passages useful and ranked early
Evaluation rows completed	6	One row per reviewed question
Published results file	written	Run accepted because every metric completed

Table 4.4: Outcome of the Ragas execution on the live system.

Table 4.4 reports the answer-level quality of the platform. A faithfulness of 0.94 means that the vast majority of the statements produced by the generator can be traced back to the retrieved context, which is the same property that the explicit refusal at the end of the example answer in Section 4.5 illustrates. A context precision of 0.89 indicates that the retriever occasionally ranks a passage that is semantically close but substantively irrelevant, the phenomenon visible in the third source of Table 4.6. Because every metric completed within the run budget, the results file is published and the evaluation view of the interface displays these values instead of reporting Ragas as unavailable.

4.4 End-to-End Validation

The components were also validated end to end against the external providers and against the pipeline itself, so that the report does not rest only on offline measurements. Table 4.5 summarises the observed evidence.

ID	Validation target	Observed evidence	Status
V-01	Heterogeneous ingestion	HTML, RSS/Atom and PDF sources parsed through one provenance-aware path; each accepted item keeps its URL, language, document type and crawl timestamp	Pass
V-02	Chunking and indexing	574 chunks for 54 documents, all 574 present in the vector index with their chunk identifiers	Pass
V-03	Production embeddings	Passage and query embeddings returned 2048-dimensional vectors from the NVIDIA endpoint	Pass
V-04	Grounded generation	Answer generated in 3.41 s from retrieved context with the source list attached	Pass
V-05	Remote entity extraction	NVIDIA extraction returned 7 entities and 3 relations on a bounded multilingual sample	Pass
V-06	Provider saturation handling	Mistral extraction returned HTTP 429 and the chunk was deferred to the durable retry queue instead of being dropped	Pass
V-07	Graph analytics	Neo4j build published 82 nodes and 355 edges; PageRank, degree centrality, communities and shortest paths exposed by the API	Pass
V-08	Answer-level evaluation	Ragas completed six rows on real retrieved context; faithfulness 0.94, answer relevancy 0.92 and context precision 0.89	Pass
V-09	Distributed ETL on a cluster	Spark and WebHDFS paths implemented and flag-gated; the reported figures were measured with the local engines	Pending

Table 4.5: End-to-end validation of the platform components.

The validation matrix separates what has been demonstrated from what remains to be demonstrated. Ingestion, chunking, indexing, embeddings, generation, extraction, provider-failure handling, graph analytics and the answer-level evaluation were all observed on the live system. One entry remains open: the distributed Spark and WebHDFS paths are implemented but the reported measurements were produced with their local equivalents, which is why they are marked as pending rather than as passed.

4.5 Example of a Grounded Answer

To make the response contract of the platform concrete, this section reproduces one complete exchange in the exact shape returned by the service: the question, the grounded answer and the sources that were attached to it. The exchange is an illustration of the

format and of the provenance mechanism, so it is not counted in the metrics of the previous sections.

Question. *Quels types de rapports financiers les émetteurs publient-ils auprès de l'AMMC ?*

Answer. According to the AMMC listing of issuers' financial statements, the published documents fall into two families that the page exposes directly. The first is the annual reports, indexed by fiscal year: the listing for the 2025 series names, among others, STOKVIS NORD AFRIQUE, MANAGEM, CTM, HOLCIM MAROC and ATTIJARIWAFA BANK. The second is the first-semester reports, published for the following fiscal year, where the 2026 series names EQDOM, Crédit Agricole du Maroc, CIH Bank, BMCI and Attijariwafa Bank. The listing can be browsed by year and by issuer. The supplied context does not provide evidence of a quarterly report family, so that report type cannot be confirmed from the indexed sources.

The answer is not a paraphrase of the question: each statement is traceable to a retrieved passage, and the last sentence is an explicit refusal to assert something the evidence does not support. The response also carried the matched entities (AMMC, Bank Al-Maghrib), the size of the traversed subgraph, the evidence snippets with their scores, the mean latency of the request and a mode field stating whether the text was generated by the provider model or composed locally by the deterministic fallback. Table 4.6 lists the sources that accompanied the answer, in the retrieval order in which the service returned them.

Rank	Source	Type and role in the answer
1	ammc.ma — liste des états financiers des émetteurs (rapports annuels)	HTML listing; supports the annual-report statement and supplies the issuer names of the 2025 series.
2	ammc.ma — même listing, rapports 1er semestre	HTML listing; supports the first-semester statement and supplies the issuer names of the 2026 series.
3	bkam.ma — Rapport annuel 2024 (PDF)	PDF retrieved for the question but unrelated to AMMC reporting obligations; correctly ignored by the answer, and one of the off-topic passages that the context-precision metric of Table 4.4 is designed to detect.

Table 4.6: Sources returned with the example answer, in retrieval order.

Two behaviours are visible in Table 4.6. The evidence list is diversified, because the service reserves at most one passage per document, so the two AMMC passages

come from two distinct chunks of the same listing rather than from a single fragment repeated; and the third source shows why provenance is useful, since a passage that the retriever judged semantically close but that is substantively irrelevant can be displayed and identified as off-topic instead of being hidden inside a fluent answer. This is exactly the situation that the Ragas context-precision metric isolates, and the example therefore shows the same phenomenon that the evaluation measures.

4.6 Discussion

The results support three conclusions and expose three gaps. The platform works end to end: a heterogeneous document collected from an authorised source reaches a cited answer, and every intermediate state preserves the provenance that makes the answer verifiable. Retrieval is functional but not yet discriminating: a hybrid vector and BM25 retriever achieves a Recall@5 of 0.75 on the reviewed cases, and adding the graph neither improves nor degrades that ranking on such a small set. The evaluation chain is operational: Ragas runs against the live system and returns a complete set of answer-level scores, with a faithfulness of 0.94, an answer relevancy of 0.92 and a context precision of 0.89.

The first gap is evaluation depth. Four navigation questions cannot distinguish two strategies whose rankings coincide; questions requiring several hops, questions written in Arabic, and questions whose answer does not exist in the corpus are needed before the recall figures can be interpreted as quality. The second gap is extraction coverage: the published build carries the heuristic provenance tag, so the relational content of the graph understates what the language-model extractor produces, as the bounded provider comparison and the live check both show. The third gap is the distributed profile: Spark and WebHDFS are implemented and gated behind flags, but the end-to-end ingestion has so far been exercised with the local equivalents, and a cluster run must be performed before the distributed claims can be stated as measured.

These three gaps also determine the order of the perspectives presented in the conclusion: build the reviewed multilingual and multi-hop question set, run the extraction pipeline at full remote coverage on the corpus, and then rerun the same benchmark on the cluster profile so that the comparison between vector and graph retrieval is made on material produced by the distributed path.

Conclusion and Perspectives

This report presented a Moroccan GraphRAG platform for source-grounded business intelligence at ANRT. The solution connects respectful HTML, RSS and PDF crawling with an HDFS-backed raw zone, a Spark ETL stage with a pure-Python equivalent, Mistral entity and relation extraction, NVIDIA embeddings of 2048 dimensions stored in Qdrant, a Neo4j property graph, BM25 hybrid retrieval, FastAPI services and a React interface. The corpus aggregates roughly 78 000 documents of different types from HCP, AMMC and Bank Al-Maghrib, of which the prototype indexes a curated working subset of 54 documents and 574 chunks.

The platform was validated through the published ingestion build, source-quality scoring, retrieval metrics and a Ragas execution path. The graph and the vector index are consistent, with one indexed vector point per chunk and 82 nodes and 355 edges in the knowledge graph. On four reviewed navigation questions, hybrid vector retrieval and GraphRAG both reach a Recall@5 of 0.75, an MRR of 0.5833 and an nDCG@5 of 0.625, at a mean latency of 74.36 ms and 94.32 ms respectively. The Ragas run confirmed that the judge pipeline works on real retrieved context and returned a complete set of answer-level scores — a faithfulness of 0.94, an answer relevancy of 0.92 and a context precision of 0.89 — so the results file was published and the evaluation view of the interface displays these values.

Three perspectives follow directly from those results. The first is evaluation depth: a reviewed multilingual question set covering Arabic, French and English, multi-hop questions and unanswerable cases is required before recall figures can be read as quality. The second is extraction coverage: the published build carries the heuristic provenance tag, and the pipeline must be run at full remote coverage so that the graph reflects the entities and relations the language model actually produces. The third is the distributed profile: Spark and WebHDFS are implemented and flag-gated, and the end-to-end ingestion must be exercised on a real cluster so that the distributed claims are measured rather than extrapolated. Adding authentication, and monitoring extraction quality separately from retrieval quality, complete the roadmap towards a production deployment.